Dhirubhai Ambani University (DA-IICT), Gandhinagar

Course IT645 – Web and Mobile Development

TaskBoard

A Kanban-based project and task tracker

Project report

Faculty	Dr. Shruti Bhilare
Live application	https://taskboard-gu4g.onrender.com
Source code	https://github.com/24Chessman/TaskBoard

Team member	**Student ID**	**Modules**
Naitik Sutariya	202512006	Tasks and Board, GitHub issue lookup
Harshal Prajapati	202512013	Authentication, Users, activity logger
Jay Trivedi	202512123	Projects and Teams
Jaydip	202512090	Comments, Activity feed, Dashboard

November 2026

Contents

1 Introduction **3**

1.1 Purpose . . . 3

1.2 Scope . . . 3

2 Requirements **3**

2.1 Functional requirements by module . . . 3

2.2 Roles and permissions . . . 4

2.3 Non-functional requirements . . . 4

3 System architecture **4**

3.1 Overview . . . 4

3.2 Technology stack . . . 5

3.3 Server request pipeline . . . 5

3.4 Repository layout . . . 5

4 Database design **6**

4.1 Collections . . . 6

4.2 Referential rules implemented in controllers . . . 6

5 API design **7**

5.1 Conventions . . . 7

5.2 Endpoints . . . 7

5.3 GitHub issue lookup . . . 8

6 Security **8**

7 Frontend **9**

7.1 Pages and navigation . . . 9

7.2 State and data loading . . . 9

7.3 Board and drag and drop . . . 9

7.4 Design system . . . 9

7.5 Accessibility and responsiveness . . . 10

7.6 Screenshots . . . 10

8 Testing and quality assurance **15**

8.1 API tests . . . 15

8.2 Continuous integration . . . 16

8.3 Manual end-to-end test 16
8.4 Build size 16

9 Deployment **16**

10 Team organisation and contributions **17**

11 Limitations and future work **17**

12 Conclusion **18**

1 Introduction

1.1 Purpose

TaskBoard is a lightweight project and task tracker, a small version of tools such as Jira. Teams create projects, invite members and manage work on a Kanban board with four columns: To Do, In Progress, Review and Done. Every task carries a lead, a priority, labels and a due date. Members discuss a task in its comments, every change is written to an activity log, and a dashboard summarises progress.

The application is a full-stack MERN project (MongoDB, Express, React, Node.js) built for the course IT645. It is deployed as a single web service and is available at the address on the title page.

1.2 Scope

The system provides:

- account registration, login and a profile page;
- projects with members and two project roles (owner and member), plus a global administrator;
- a board with drag-and-drop between columns by mouse, touch and keyboard;
- tasks with title, description, status, priority, lead, labels, due date and an optional link to a GitHub issue or pull request;
- comments on tasks, an append-only activity feed and a per-project dashboard;
- an administration page for users;
- a responsive interface that works from 360 px phone width upwards.

Out of scope are real-time updates, file attachments, notifications, several leads per task and ordering of cards inside a column.

2 Requirements

2.1 Functional requirements by module

Module	Capabilities
Authentication and users	Register, log in, current user; edit own name, email and password (current password required); search users; administrators change roles and delete users.
Projects and teams	Create, edit, archive and delete projects; add and remove members; the creator becomes owner; the last owner cannot be removed.
Tasks and board	Create, edit, delete tasks; change status; filter by lead, priority, label and search text; sort; drag and drop; GitHub issue link with title and open or closed state.
Comments	Add, edit and delete comments on a task.
Activity feed	Read-only history of project, member, task and comment events, newest first.

Module	Capabilities
Dashboard	Task totals, tasks by status and priority, overdue and due-soon counts, tasks without a lead, workload per member, top labels, tasks created per day.

2.2 Roles and permissions

There are two layers. The global role of a user is `user` or `admin`. Inside a project a member is `owner` or `member`. A global administrator counts as owner of every project.

Action	Allowed for
Register, log in	anyone
Create a project	any logged-in user (becomes owner)
View a project, its tasks, comments, activity, dashboard	project members, administrator
Edit, archive, delete a project; manage members	project owner
Create a task; change a task status; add a comment	any project member
Edit a task	owner, the task creator, or the task lead
Delete a task	owner or the task creator
Edit a comment	its author only
Delete a comment	its author or the project owner
Change a user role; delete a user	administrator only

Table 2: Permission rules enforced by the server.

2.3 Non-functional requirements

- **Security:** hashed passwords, token-based authentication, server-side authorisation on every route, validated input.
- **Usability:** responsive layout from 360 px, keyboard access for the board and all menus, plain error messages.
- **Maintainability:** one folder per concern, files kept short, a written API contract and a design system document.
- **Deployability:** one build command and one start command; configuration only through environment variables.

3 System architecture

3.1 Overview

The browser loads a single-page React application. The application talks to a REST API built with Express. The API stores data in MongoDB Atlas through Mongoose and, for the issue lookup, calls the public GitHub API. In production one Node.js process serves both the API under `/api` and the built client files, so the application and the API share one origin and no cross-origin configuration is needed.

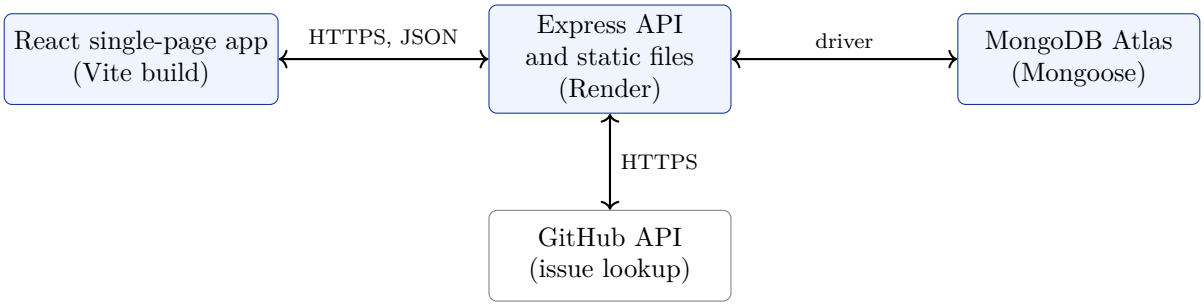


Figure 1: Runtime architecture.

3.2 Technology stack

Versions are the ones installed from the lock files when this report was prepared.

Layer	Technology	Version
Runtime	Node.js	18 or newer (22.19 used for testing)
Server	Express	4.22.3
Server	Mongoose	8.24.5
Server	jsonwebtoken, bcryptjs	9.0.3, 3.0.3
Client	React, React Router	18.3.1, 6.30.6
Client	Vite, Tailwind CSS	8.3.5, 3.4.19
Client	axios, Recharts, lucide-react	1.21.0, 3.10.1, 1.66.0
Client	@dnd-kit/core (drag and drop)	6.3.1
Database	MongoDB Atlas	hosted
Hosting	Render web service	free tier
Quality	Postman collections, Newman 6.2.3, GitHub Actions	–

Table 3: Technology stack.

3.3 Server request pipeline

Each request passes through the same chain in `app.js`: security headers, CORS policy, request logging, JSON body parsing, then the route. Routes add only the middleware they need: `auth` (valid token), `projectAccess` (membership), `requireProjectOwner`, `requireRole` and `activityLogger`. After the API routes, in production, a static handler serves the client; after that come the not-found handler and one central error handler that converts Mongoose validation errors, duplicate keys and bad identifiers into the standard error response.

3.4 Repository layout

- `server/src`: `controllers` (8 files), `middleware` (9), `models` (5), `routes` (8), `config`, `scripts`, `utils`.
- `client/src`: `api`, `components` (grouped by feature), `context`, `hooks` (8), `pages` (14), `routes` (4 guards), `utils`.
- `docs`: product requirements, technical requirements, application flow, API contract, design system, eight Postman collections.
- `.github/workflows`: the continuous-integration workflow.

4 Database design

The database has five collections. Timestamps (`createdAt`, `updatedAt`) are kept on all of them.

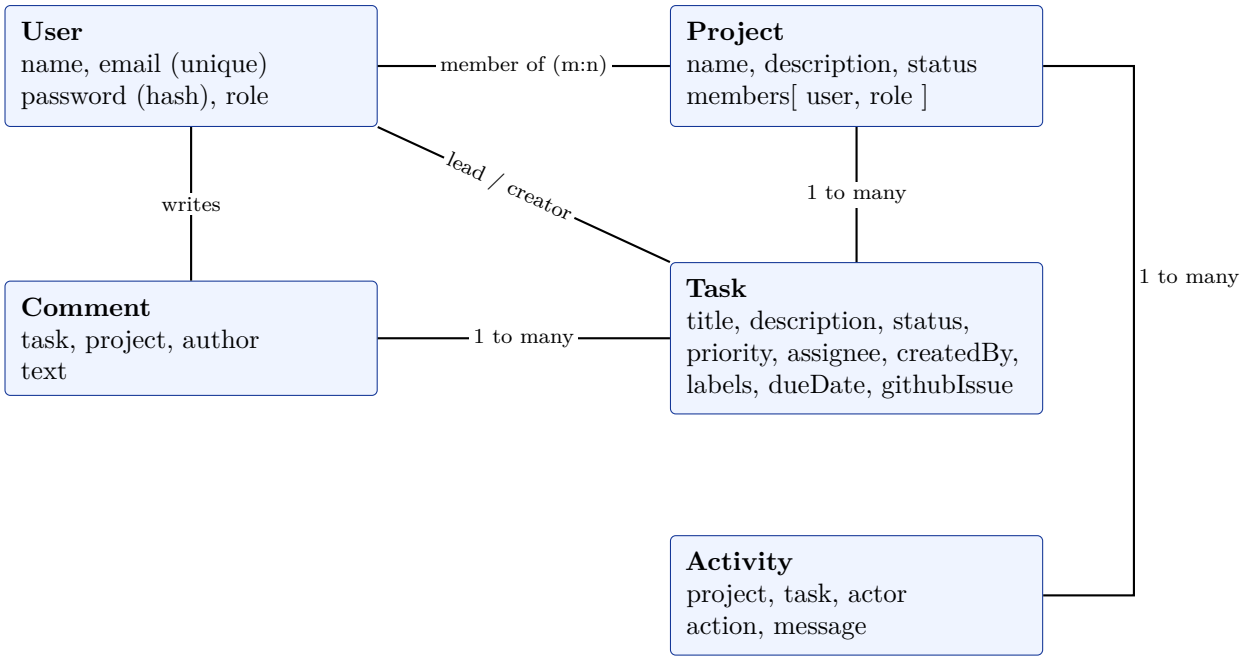


Figure 2: Entity relationships.

4.1 Collections

Model	Fields and rules
User	`name` (2–50 characters); `email` (required, unique, lower case, pattern-checked); `password` (minimum 6 characters, never returned by queries, hashed with bcrypt, 10 rounds); `role` (`user` or `admin`).
Project	`name` (3–80); `description` (up to 500); `status` (`active` or `archived`); `members`, an array of `{user, role}` where a user appears at most once and at least one member is an owner. Index on `members.user`.
Task	`project`; `title` (1–120); `description` (up to 2000); `status` (`todo`, `in_progress`, `review`, `done`); `priority` (`low`, `medium`, `high`); `assignee` (optional, shown as "Lead" in the interface, must be a project member); `createdBy`; `labels` (at most 5, 1–20 characters each); `dueDate`; `githubIssue`, an embedded document with `url`, `title`, `state`. Indexes on project with status and project with due date.
Comment	`task`, `project`, `author`, `text` (1–1000). Index on task and creation time.
Activity	`project`, optional `task`, `actor`, `action` (nine values such as `task.created` or `comment.added`) and a readable `message` stored when the event is written. Append-only: no update or delete endpoint exists. Index on project and time.

4.2 Referential rules implemented in controllers

MongoDB does not enforce foreign keys, so the controllers do:

- Deleting a project deletes its tasks, comments and activity.

- Deleting a task deletes its comments.
- Deleting a user clears them as lead and removes them from projects, but keeps their comments and activity, which are then shown as "Deleted user". A user who is the only owner of a project cannot be deleted.
- Removing a member clears them as lead in that project.

5 API design

The API lives under `/api`, uses JSON, and has 29 endpoints plus a public health check. The full contract is in `docs/backend_schema.md`.

5.1 Conventions

- Success: `{ success: true, data }`; lists add `page`, `limit` and `total`.
- Error: `{ success: false, message, errors: [{ field, message }] }`.
- Status codes: 200, 201, 204 (delete, no body), 400 (validation, bad identifier), 401, 403, 404, 409 (duplicate), 429 and 502 (GitHub lookup).
- Paging: projects default 10 (maximum 50), tasks 50 (100), comments 50 (100), activity 10 (50).
- Task list filters: `status`, `priority`, `assignee`, `label`, `dueBefore`, `search`, `sort`, `page`, `limit`.

5.2 Endpoints

#	Method	Path	Access	Purpose
1	POST	`/auth/register`	public	create account
2	POST	`/auth/login`	public	log in
3	GET	`/auth/me`	login	current user
4	POST	`/projects`	login	create project
5	GET	`/projects`	login	list projects
6	GET	`/projects/:projectId`	member	one project
7	PUT	`/projects/:projectId`	owner	update project
8	PATCH	`/projects/:projectId/status`	owner	archive, reactivate
9	DELETE	`/projects/:projectId`	owner	delete project
10	POST	`/projects/:projectId/members`	owner	add member
11	DELETE	`/projects/:projectId/members/:userId`	owner	remove member
12	POST	`/projects/:projectId/tasks`	member	create task
13	GET	`/projects/:projectId/tasks`	member	list tasks
14	GET	`/projects/:projectId/tasks/:taskId`	member	one task
15	PUT	`/projects/:projectId/tasks/:taskId`	owner, creator, lead	update task
16	PATCH	`/projects/:projectId/tasks/:taskId/status`	member	change status
17	DELETE	`/projects/:projectId/tasks/:taskId`	owner, creator	delete task

#	Method	Path	Access	Purpose
18	POST	`/projects/:projectId/tasks/:taskId/comments`	member	add comment
19	GET	`/projects/:projectId/tasks/:taskId/comments`	member	list comments
20	PUT	`/projects/:projectId/tasks/:taskId/comments/:commentId`	author	edit comment
21	DELETE	`/projects/:projectId/tasks/:taskId/comments/:commentId`	author, owner	delete comment
22	GET	`/projects/:projectId/activity`	member	activity feed
23	GET	`/projects/:projectId/dashboard`	member	statistics
24	GET	`/users`	login	search users
25	GET	`/users/:userId`	login	public profile
26	PUT	`/users/:userId`	self, admin	update profile
27	PATCH	`/users/:userId/role`	admin	change role
28	DELETE	`/users/:userId`	admin	delete user
29	GET	`/github/issue?url=`	login	look up issue
30	GET	`/health`	public	service status

5.3 GitHub issue lookup

A task can carry a link to a GitHub issue or pull request. The client sends the link to `/github/issue`; the server parses it strictly, builds the request to `api.github.com` from the parsed owner, repository and number, and returns only the title and the state (open or closed). Only `https` links on `github.com` with a path of the form `/owner/repo/issues/N` or `/pull/N` are accepted. The request has a five-second timeout, redirects are never followed, successful answers are cached in memory (100 entries, 5 minutes), and an optional server-side token raises the GitHub rate limit. On save the server validates the link, the title (up to 200 characters) and the state.

6 Security

- **Passwords** are hashed with bcrypt (10 rounds) in a pre-save hook and are excluded from queries and JSON output.
- **Tokens:** a JWT signed with a secret from the environment carries only the user identifier and expires after 7 days by default. The client keeps it in `localStorage` and sends it as a Bearer header; any 401 clears it and returns the user to the login page. The server rejects missing, malformed, expired and invalid tokens and tokens of deleted users.
- **Authorisation** is checked on the server for every route, in addition to the guards in the interface.
- **Input validation:** identifiers, enumerations, lengths and types are checked before saving; query values must be plain strings, which blocks operator injection.
- **Headers:** `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy` and `Permissions-Policy` are set on every response; `X-Powered-By` is disabled.
- **CORS:** in production no cross-origin headers are sent unless origins are listed in the optional variable `CLIENT_ORIGIN`.
- **Start-up checks:** in production the server refuses to start when `MONGO_URI` is missing or `JWT_SECRET` is missing, shorter than 32 characters or looks like a placeholder. The message

names the variable and never prints its value.

7 Frontend

7.1 Pages and navigation

Route	Page
`/`	landing page for visitors
`/login`, `/register`	authentication
`/projects`, `/projects/new`	project list, new project
`/projects/:projectId`	board
`/projects/:projectId/dashboard`	dashboard (loaded on demand)
`/projects/:projectId/activity`	activity feed
`/projects/:projectId/settings`	project settings and members (owner only)
`/profile`	profile and password
`/admin/users`	user administration (admin only)

Table 6: Routes of the client application.

Three route guards protect the pages: one requires a login, one keeps logged-in users away from the login and register pages, and one shows the Forbidden page unless the user is an administrator. A member who opens the settings address of a project also sees the Forbidden page.

7.2 State and data loading

The client uses no state library. Authentication and toast messages live in React context. A `useFetch` hook loads data and keeps the previous data visible while refreshing. Board filters live in the URL, so a filtered board can be shared and survives a reload. A single helper maps network failures and HTTP statuses to plain messages, so raw error text never reaches the screen.

7.3 Board and drag and drop

The board uses `@dnd-kit/core`. A card is picked up by mouse (after 8 px of movement), by touch (after a 200 ms press, so normal swipes still scroll the board) or by keyboard: Space to pick up, arrow keys to change column, Space to drop, Escape to cancel. The move is shown at once and saved with a status request; if the request fails the card returns to its column and an error message appears. Screen readers receive announcements that name the columns. The task drawer offers a status field as an alternative to dragging.

7.4 Design system

The interface follows a written design system (`docs/design.md`): a cobalt primary colour (#1D4ED8), IBM Plex Sans with IBM Plex Mono for identifiers and counters, slate neutrals, borders instead of shadows, consistent corner radii, icons from one library, no gradients and no emoji. Only one menu or popover is open at a time and Escape closes the top layer only.

7.5 Accessibility and responsiveness

Each route sets its own page title. Dialogs and drawers trap and restore focus; icon-only buttons have names; form errors are linked to their fields; animation is reduced when the user asks for it. Text contrast was measured on 123 text and background combinations across nine pages, with a lowest ratio of 4.55:1 against the 4.5:1 requirement. Tap targets are at least 40 px on phones and the page does not scroll sideways at 360 px.

7.6 Screenshots

The screenshots show the application with a small example project (two users, eight tasks).

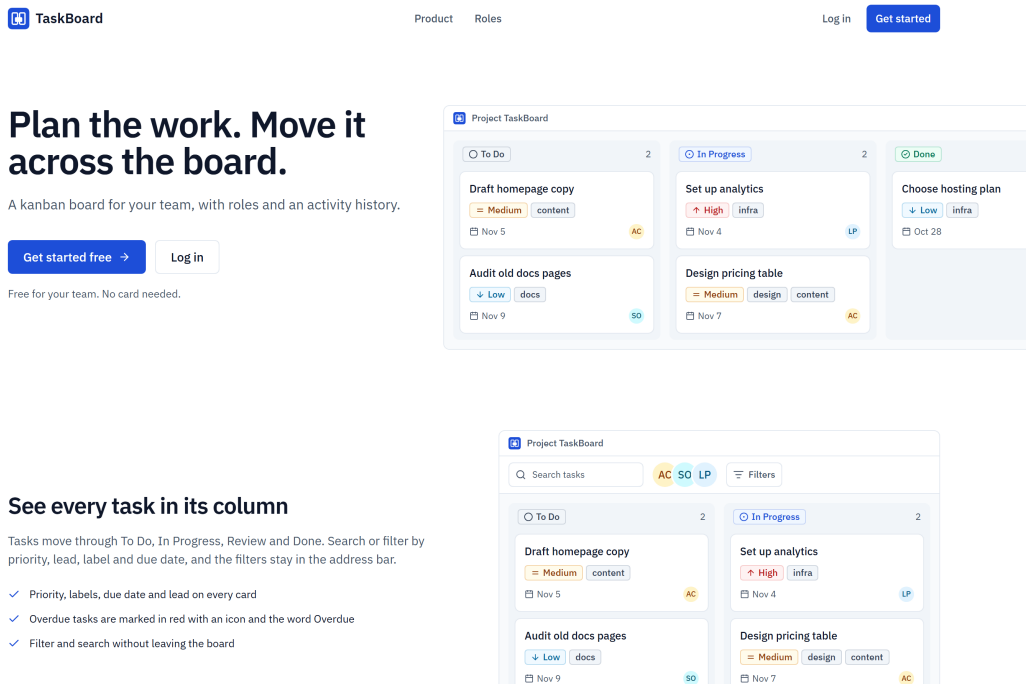
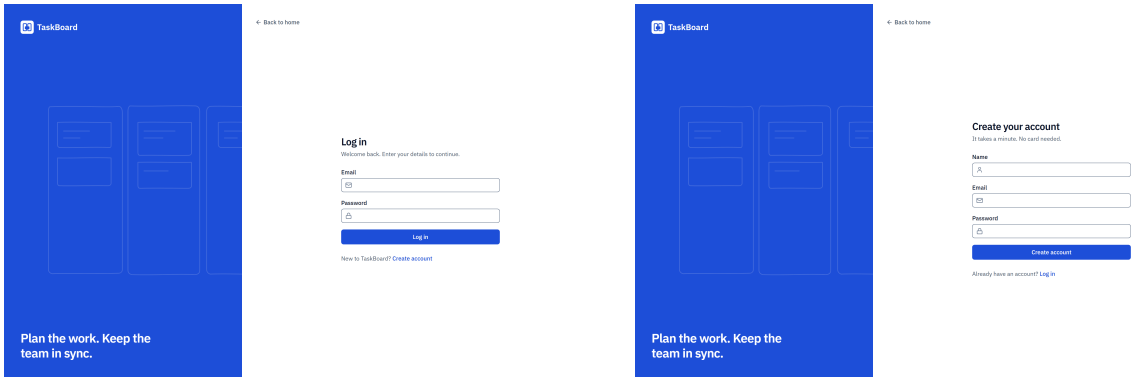


Figure 3: Landing page.



(a) Log in **(b)** Create account

Figure 4: Authentication pages.

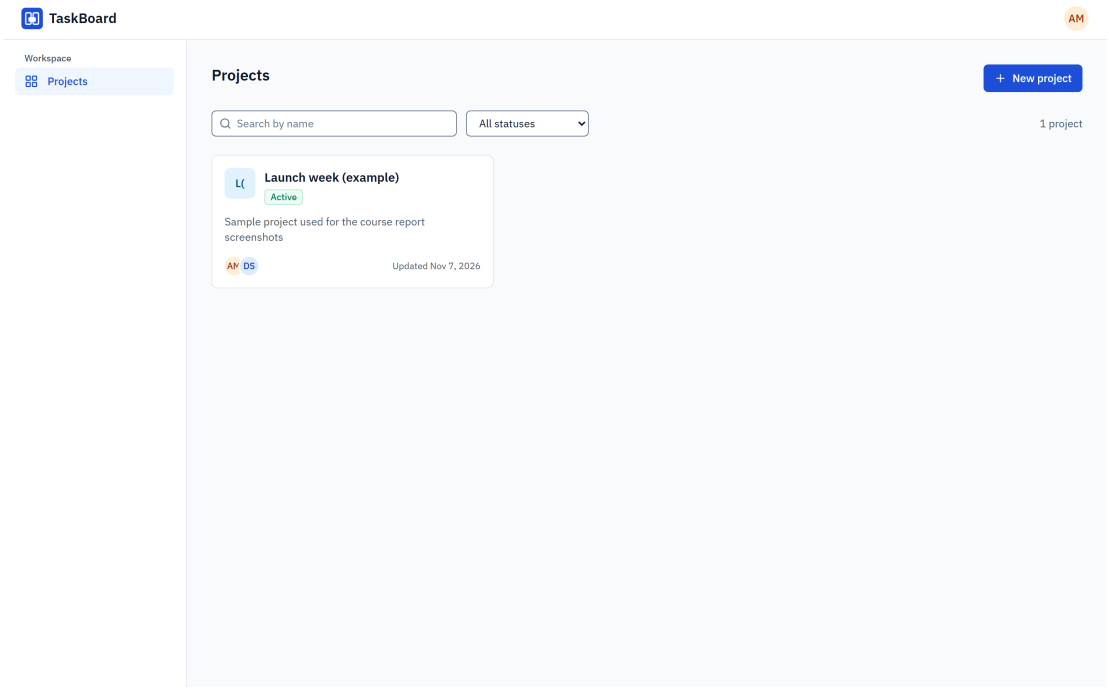


Figure 5: Project list.

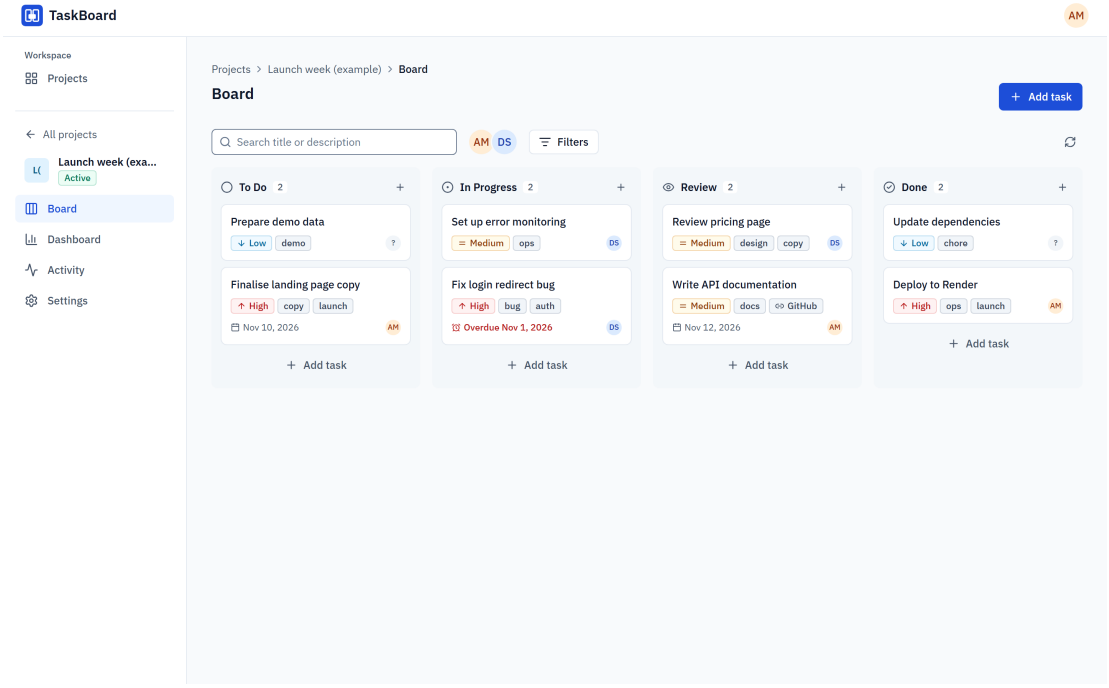
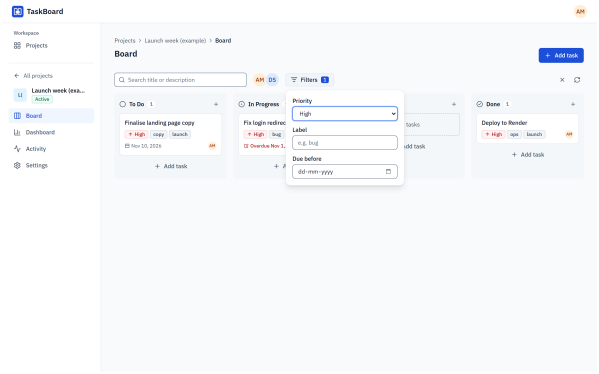
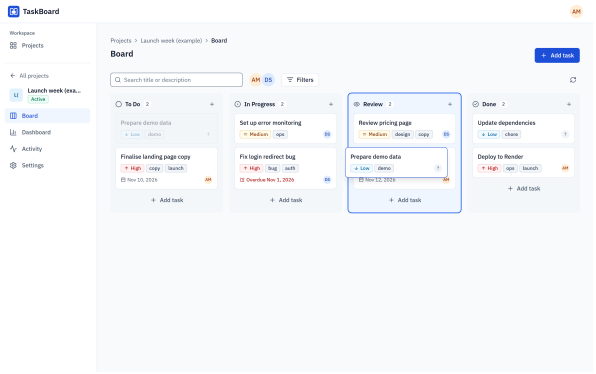


Figure 6: Board with four columns, priorities, labels, due dates and leads.

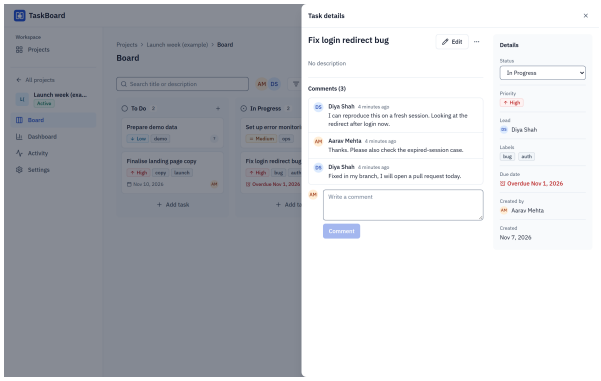


(a) Filters popover

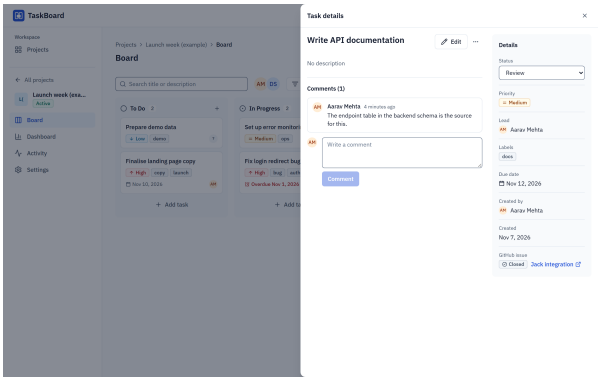


(b) Card lifted while dragging

Figure 7: Board filters and drag and drop. While a card is dragged the target column is highlighted.



(a) Details and comments



(b) Linked GitHub issue

Figure 8: Task drawer.

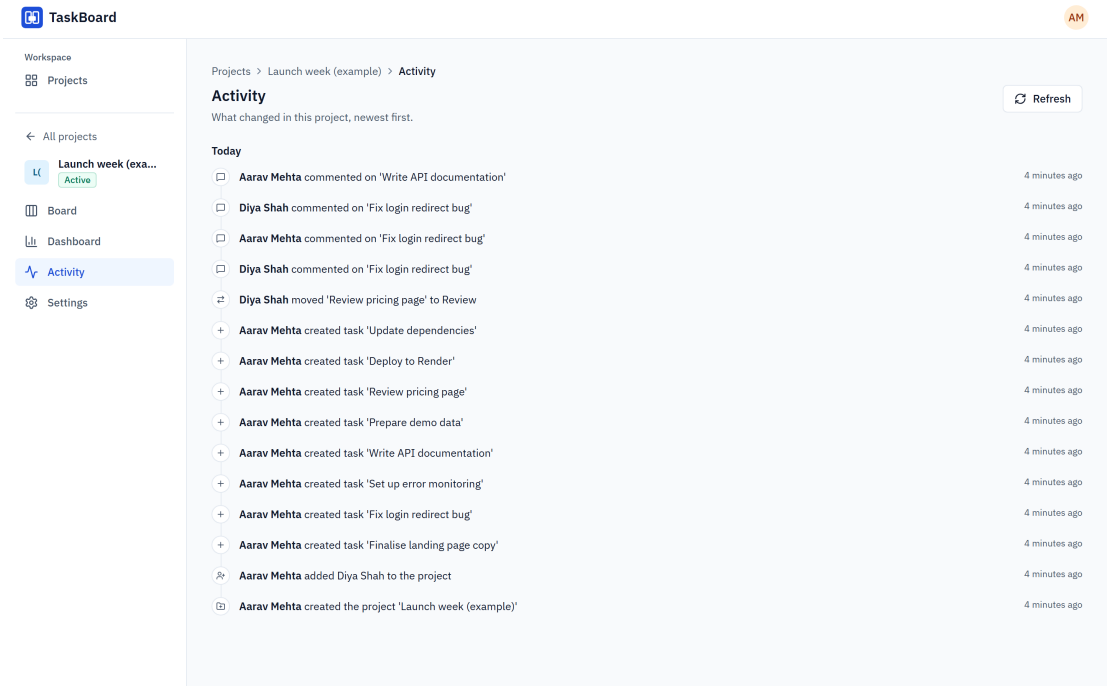


Figure 9: Activity feed, newest first.

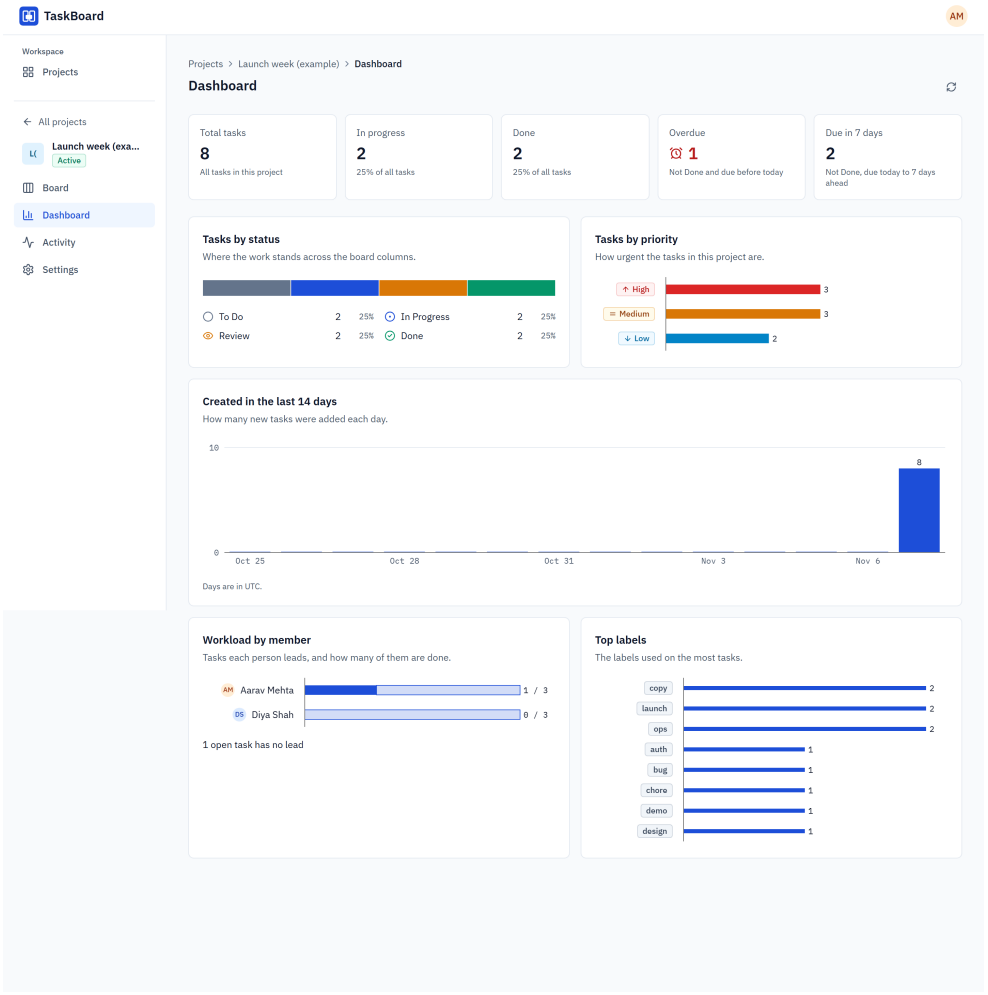
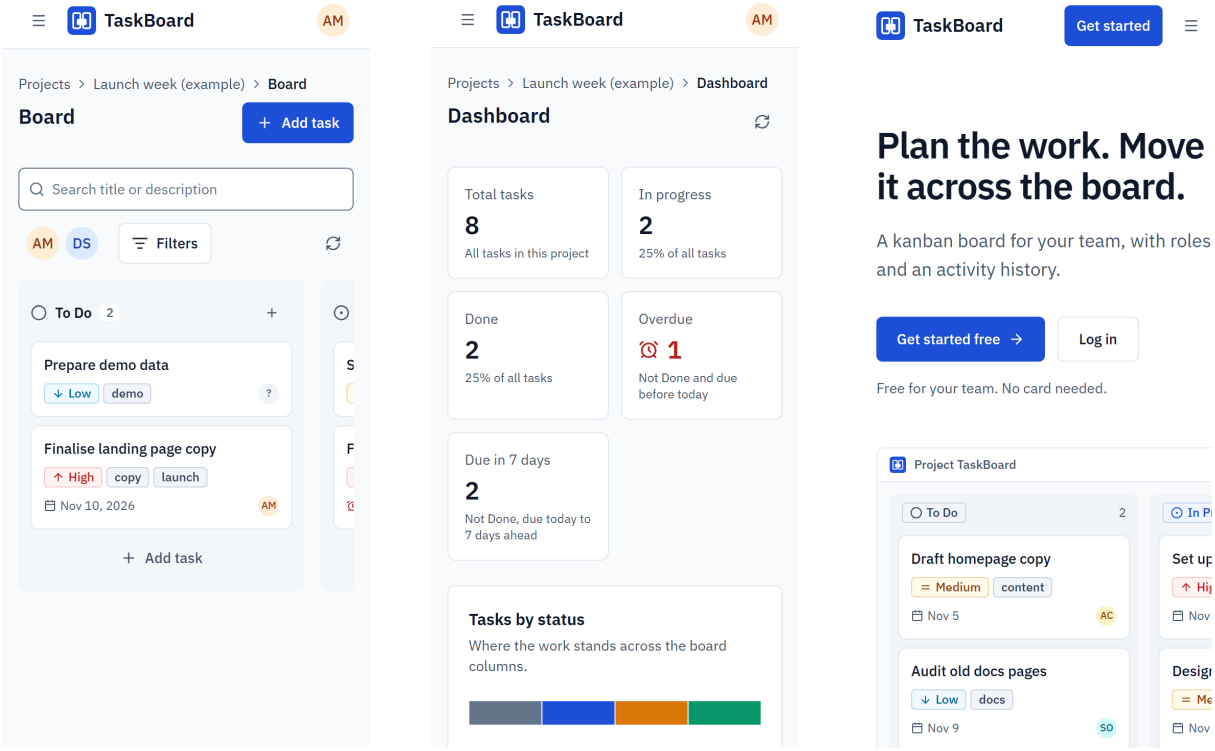


Figure 10: Project dashboard.



(a) Board **(b)** Dashboard **(c)** Landing page

Figure 11: The same pages at a 360 px phone width (the dashboard and landing page are cropped to their upper part).

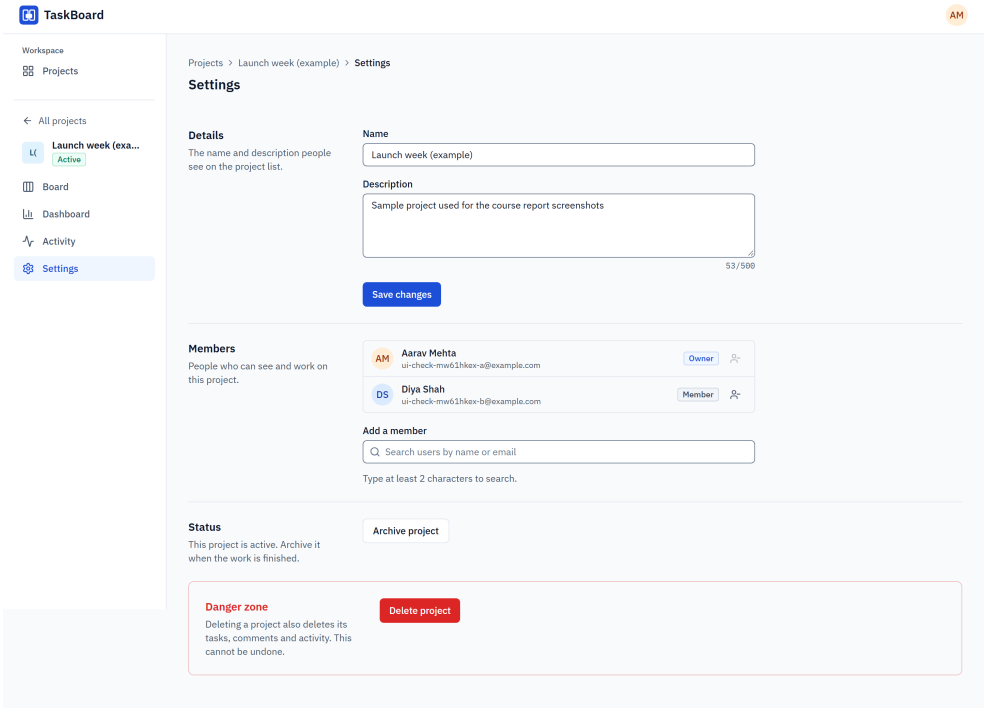
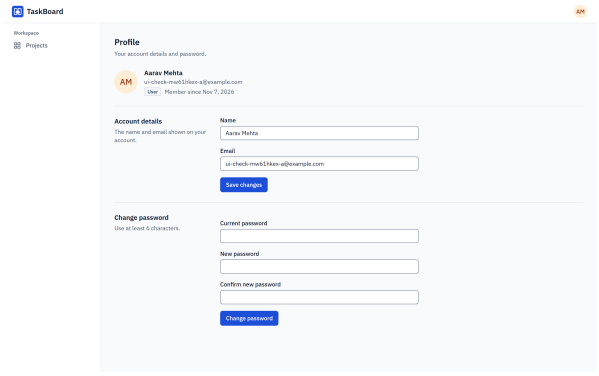
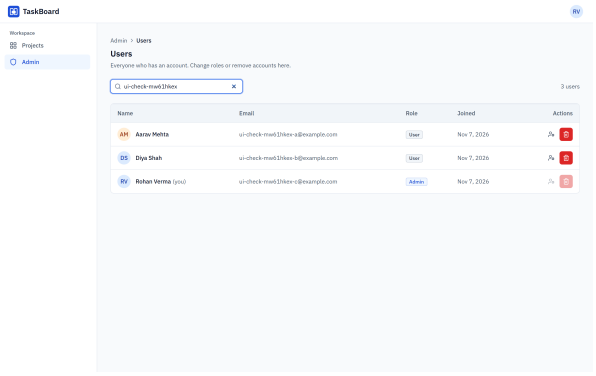


Figure 12: Project settings and member management (owner).

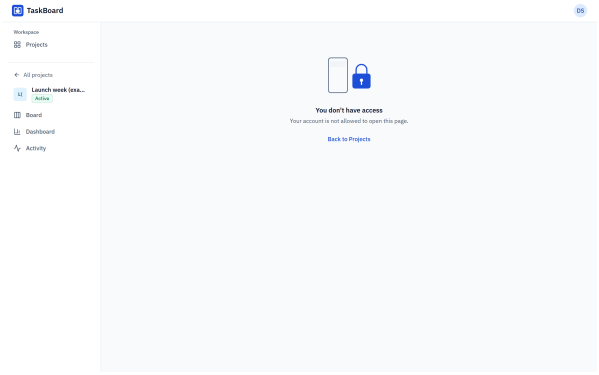


(a) Profile

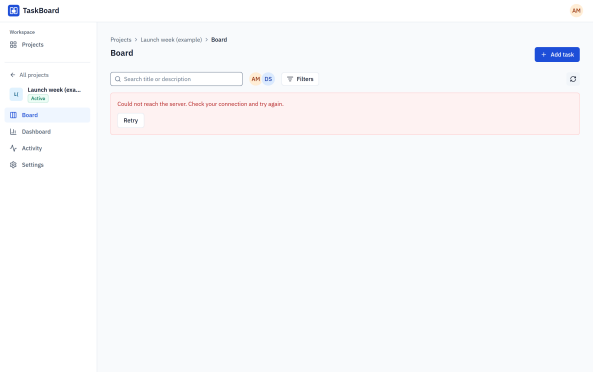


(b) User administration (administrator)

Figure 13: Account and administration pages.



(a) Forbidden page for a member



(b) Server not reachable

Figure 14: Access control and error handling.

8 Testing and quality assurance

8.1 API tests

The API is tested with eight Postman collections, one per resource, run from the command line with Newman. All requests and assertions passed against the local server on the day this report was prepared.

Collection	Requests	Assertions	Failed
auth	8	14	0
users (with the administrator folder)	24	34	0
projects	21	28	0
tasks	26	35	0
comments	29	35	0
activity	15	19	0
dashboard	17	30	0
github	18	31	0
Total	**158**	**226**	**0**

Table 7: Newman results.

8.2 Continuous integration

A GitHub Actions workflow runs on every pull request and every push to the main branch. One job builds the client. A second job starts a MongoDB container, installs the server, syntax-checks every server file, builds the client, starts the server in production mode with a secret generated for that run, checks the health endpoint, the landing page and the JSON 404, and runs the authentication and task collections. The workflow uses only official actions and no repository secrets.

8.3 Manual end-to-end test

Twelve scripted steps were run in a real Chromium browser against the local application, at desktop and 360 px width: landing page, registration and login, project and task creation, board filters and drag and drop with mouse and keyboard, task drawer and comments, GitHub issue link, dashboard figures cross-checked against the API, member rights, profile changes, administration, deletion cascades, deep links with back and forward navigation, and an offline simulation. All steps passed. During the run, a few defects in the test scripts were corrected; the application itself needed only a wording change in two server messages.

8.4 Build size

The production build produces a main script of 400.98 kB (127.32 kB compressed) and a separate 336.52 kB dashboard chunk (98.81 kB compressed) that loads only when the dashboard is opened.

9 Deployment

The application is deployed as one Render web service connected to the main branch of the repository; every merge to main redeploys it. The database is a MongoDB Atlas cluster whose network access allows the host.

Setting	Value
Root directory	repository root
Build command	`npm run render-build` (installs server and client dependencies and builds the client)
Start command	`npm start` (runs `node src/server.js`)
Health check	`/api/health`
Node version	18 or newer

Table 8: Hosting settings.

Configuration uses environment variables only: `NODE_ENV`, `MONGO_URI` and `JWT_SECRET` are required; `JWT_EXPIRES_IN`, `CLIENT_ORIGIN` and `GITHUB_TOKEN` are optional; the host provides `PORT`. The first administrator is created with `npm run seed:admin`, which reads `ADMIN_NAME`, `ADMIN_EMAIL` and `ADMIN_PASSWORD`. The production server serves hashed files under `/assets/` with a one-year cache and the page shell without caching; unknown page addresses return the application shell and unknown files return a plain 404.

10 Team organisation and contributions

Work was split by module, with each member responsible for the backend and the interface of their modules. Changes were made on feature branches and merged by pull request with a merge commit, which gave 37 merged pull requests between 15 October and 7 November 2026. The repository holds 135 non-merge commits.

Member	Modules	Commits	Main areas touched
Naitik Sutariya	Tasks and Board, GitHub lookup	68	board, task drawer, landing page, shell, GitHub URL parser, task controller
Harshal Prajapati	Authentication, Users, activity logger	25	auth and profile pages, route guards, auth and role middleware, activity logger
Jay Trivedi	Projects and Teams	20	project pages, member settings, project model and access middleware
Jaydip	Comments, Activity, Dashboard	22	dashboard charts, comment section, activity feed

Table 9: Commits by author name recorded in the repository (non-merge commits on main; counts for the first member combine three identities of the same person).

The numbers show where changes were recorded and are not a measure of effort. The code base has about 1 980 lines in 42 server files and 7 720 lines in 146 client files, with about 5 500 lines of documentation and test collections.

11 Limitations and future work

- The API has no rate limiting, and no Content-Security-Policy header is sent because the interface loads web fonts and uses inline styles.
- The GitHub lookup does not follow redirects on purpose, so a repository that GitHub has renamed or moved (for example the React repository) is reported as unreachable. Following exactly one redirect to `api.github.com` would fix this.
- The lookup cache is held in server memory and is lost when the service restarts.
- A task has one lead; several assignees per task would need a change of the data model.
- Cards cannot be ordered inside a column; only the column is stored.
- There is no “disable user” function, only deletion.
- The free hosting tier sleeps when idle, so the first request after a pause is slow.
- The client has no automated unit tests; the API is covered by the Postman collections and the interface by the manual end-to-end test.

Natural next steps are real-time updates, notifications, ordering inside columns, several leads per task and automated browser tests in the CI workflow.

12 Conclusion

TaskBoard meets the goals set at the start: teams can create projects, organise tasks on a Kanban board with drag and drop, discuss and track them, and see progress on a dashboard. The server enforces roles and validates all input, the API is covered by 158 requests and 226 assertions that all pass, the interface works from a phone upwards and from the keyboard, and the whole system builds and deploys from the repository with one command each. The remaining limitations are listed above and none of them blocks the intended use.